BFMC 2026 – Phase Status Report

Date: March 8, 2026
Prepared by: Team OPTINX
Reporting Period: February 2, 2026 – March 8, 2026
GitHub Repository: https://github.com/mahatosumit/BFMC_2026

1. Overview of Project Status

During the current reporting period, Team OPTINX concentrated on transitioning from basic system integration to stable closed-loop autonomous driving in preparation for the Qualification Round. A key milestone was the successful integration of the perception pipeline running on the Raspberry Pi 5 with the low-level actuation system controlled by the STM32 microcontroller. This integration enabled the vehicle to interpret visual inputs and generate real-time steering and speed commands.

The primary focus of this phase has been achieving reliable lane-keeping performance and consistent reactions to critical traffic signs. Extensive testing and tuning were conducted to reduce steering oscillations, calibrate perception thresholds, and minimize latency between perception and actuation. These improvements allowed the vehicle to navigate the track more consistently and perform stable runs under varying conditions.

Alongside algorithmic improvements, additional effort was invested in building a comprehensive monitoring interface. A custom Tkinter dashboard was developed to provide real-time telemetry and perception visualization, enabling the team to observe lane detection, traffic sign recognition, and vehicle state during testing.

2. Current Phase

The current development phase prioritizes system reliability and consistent environmental interaction. The vehicle architecture follows a distributed design in which computational tasks are divided between two main processing units. The Raspberry Pi 5 performs computer vision, perception, and behavioral decision making, while the STM32 microcontroller executes deterministic control of steering and motor actuation.

The perception system relies on two complementary pipelines. The first pipeline processes frames captured from the CSI camera using classical computer vision techniques. Through a perspective transformation, the image is converted into a Bird's Eye View representation of the track. Lane markings are extracted through adaptive thresholding and tracked using a sliding-window polynomial fitting method to estimate the road curvature and lane center. The second perception pipeline employs a quantized YOLOv8 model to detect semantic elements such as traffic lights, pedestrians, and road signs. These detections are processed asynchronously and fed into a rule-based decision module that governs vehicle behavior.

The outputs of these perception modules are integrated within the control layer. Steering commands are calculated using a Stanley Kinematic Controller based on cross-track error and heading deviation, while longitudinal control is governed by a state-machine-based Traffic Decision Engine that adjusts vehicle speed according to detected traffic elements. To facilitate debugging and evaluation, a custom dashboard was developed that displays BEV lane visualizations, object detections, and vehicle telemetry in real time. This interface has significantly accelerated testing and parameter tuning during track experiments.

3. Task Status – Overview

During this reporting period, major progress was achieved across the perception, control, and integration layers of the autonomous driving system.

The lane detection module underwent extensive refinement. Temporal smoothing using an Exponential Moving Average was introduced to stabilize polynomial lane estimates and reduce steering jitter. Additionally, a dead-reckoning fallback mechanism was implemented using IMU data and visual odometry to maintain trajectory estimates when lane boundaries temporarily disappear.

The semantic perception module reached a stable deployment stage. A lightweight YOLOv8 model was trained and optimized for real-time inference on the Raspberry Pi. The model operates within a dedicated asynchronous thread to prevent blocking the main control loop. Detected objects such as stop signs, pedestrians, and traffic lights are translated into behavioral commands through a rule-based decision engine.

Vehicle control performance was also significantly improved. The parameters of the Stanley controller were tuned through repeated testing to eliminate oscillatory behavior and ensure smooth convergence to the lane center. A supplementary boundary-protection mechanism was implemented to prevent the vehicle from drifting too close to track edges when detection confidence drops.

Finally, system integration tasks were completed to enable reliable communication between software and hardware components. A Python daemon manages serial communication with the STM32 microcontroller, while telemetry data including speed, steering angle, and system state is continuously logged and transmitted through the BFMC network interface.

Sustainability Considerations:

To support repeated testing while minimizing material waste, the validation track used by the team was conducted primarily from scraped and repurposed materials available in the laboratory. Reused boards and surfaces were refurbished and repainted to recreate BFMC lane markings for experimentation.

4. Blocking Points and Encountered Issues

Several challenges emerged during integration and testing. The most significant limitation was computational load on the Raspberry Pi 5 when running both the lane detection pipeline and the YOLO neural network simultaneously, occasionally introducing latency spikes. This was mitigated through model quantization and isolating inference in a dedicated processing thread.

Camera stability also required attention. Variations in lighting conditions initially affected lane segmentation accuracy. To improve robustness, camera exposure parameters are calibrated during initialization and then locked, making the perception pipeline more resilient to moderate lighting changes.

Communication between the Raspberry Pi and the STM32 microcontroller required tuning, as differences in processing frequency initially caused buffer overruns and delayed actuation. This was resolved by implementing a heartbeat synchronization protocol and optimizing the command parser on the microcontroller firmware.

Steering stability was further improved by combining perception-based lane estimation with IMU yaw feedback, enabling smoother trajectory correction while ensuring the vehicle consistently follows the right-hand lane as required by BFMC rules.

5. Next Steps

With the qualification deadline approaching, the immediate priority is to perform final validation runs on the physical track to ensure stable lane-keeping behavior and reliable stop-sign responses. The system will undergo repeated end-to-end testing to confirm that perception, decision making, and control modules operate consistently under real conditions.

Following qualification, development will focus on expanding the vehicle's behavioral capabilities for the competition finals. Planned improvements include refining the parking state machine, improving intersection handling logic, and further optimizing the perception pipeline for dynamic obstacles such as pedestrians and moving vehicles.

Additional work will focus on improving the temporal consistency of object detection and reducing perception latency. Future development will also explore dead-reckoning based control, adaptive lane changing through path planning, and visual mapping with localization to enhance navigation robustness.